



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

UNIVERSITÀ DEGLI STUDI DI FIRENZE  
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

---

## Laboratorio di Optimization Methods

---

*Autore:*  
Niccolò Scommegna

*Corso principale:*  
Optimization Methods

*N° Matricola:*  
7162797

*Docente corso:*  
Matteo Lapucci

17 novembre 2025

## Indice

|          |                                                              |          |
|----------|--------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduzione</b>                                          | <b>2</b> |
| <b>2</b> | <b>Nozioni Teoriche</b>                                      | <b>2</b> |
| <b>3</b> | <b>Esperimenti</b>                                           | <b>4</b> |
| 3.1      | Numero di Iterazioni . . . . .                               | 4        |
| 3.2      | Andamento Funzione Obiettivo e Norme del Gradiente . . . . . | 4        |
| 3.3      | Tempi di Esecuzione . . . . .                                | 4        |

## 1 Introduzione

In questo report vengono presentati i risultati ottenuti durante lo svolgimento del laboratorio di *Optimization Methods*, che ha avuto come obiettivo l'implementazione dell'algoritmo proposto nell'articolo *A Noise-Tolerant Quasi-Newton Algorithm for Unconstrained Optimization* [1] e il suo confronto con altri algoritmi di ottimizzazione non vincolata, come l'algoritmo di discesa del gradiente e l'algoritmo BFGS. L'implementazione è stata effettuata in linguaggio `Python` e i problemi di ottimizzazione sono stati selezionati dalla libreria `PyCUTEst`.

## 2 Nozioni Teoriche

Gli algoritmi per l'ottimizzazione non vincolata mirano a trovare il minimo di una funzione obiettivo  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  utilizzando informazioni sulla funzione stessa, come il gradiente e la matrice Hessiana. In questa sezione vengono descritte le nozioni teoriche fondamentali alla base degli algoritmi implementati nel progetto, in particolare l'algoritmo di discesa del gradiente, l'algoritmo BFGS e l'algoritmo Quasi-Newton tollerante al rumore proposto da Shi et al. [1]. Il metodo di discesa del gradiente utilizza come direzione di discesa l'antigradiente della funzione obiettivo  $-\nabla f(x_k)$  e richiede la scelta di uno scalare  $\alpha_k$  per determinare la lunghezza del passo. Una pratica comune è la ricerca di  $\alpha_k$  tramite line search, che può essere eseguita utilizzando condizioni di Armijo o Wolfe. Queste condizioni garantiscono che il passo scelto riduca sufficientemente il valore della funzione obiettivo e che la variazione del gradiente lungo la direzione di discesa sia compatibile con la curvatura locale della funzione. In notazione tipica, la condizione di Armijo si scrive come:

$$f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^T d_k, \quad (1)$$

dove  $c_1 \in (0, 1)$  è una costante predefinita e  $d_k$  è la direzione di discesa. Per quanto riguarda le condizioni di Wolfe, esse includono sia la condizione di Armijo che una condizione di curvatura. Nel complesso le condizioni forti di Wolfe si esprimono come:

$$f(x_k + \alpha_k d_k) \leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^T d_k, \quad (2)$$

$$|\nabla f(x_k + \alpha_k d_k)^T d_k| \leq c_2 |\nabla f(x_k)^T d_k|, \quad (3)$$

dove  $c_2 \in (c_1, 1)$  è un'altra costante predefinita.

L'algoritmo BFGS è un metodo Quasi-Newton che costruisce un'approssimazione della matrice Hessiana  $B_k$  basata sui gradienti calcolati nei punti precedenti e la differenza tra i punti delle iterazioni. La matrice  $B_k$  viene aggiornata ad ogni iterazione utilizzando la formula di aggiornamento di BFGS:

$$B_{k+1} = B_k + \frac{y_k y_k^T}{s_k^T y_k} - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k}, \quad (4)$$

dove  $s_k = x_{k+1} - x_k$  e  $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$ . Perché l'aggiornamento sia ben definito, e soprattutto per garantire che  $B_{k+1}$  sia definita positiva, è necessario e sufficiente che  $s_k^T y_k > 0$ . Le condizioni di Wolfe forti sono tipicamente impiegate

per assicurare questa proprietà e quindi preservare la definita positività della matrice approssimata della Hessiana ad ogni iterazione. Senza una corretta scelta del passo, che soddisfi tali condizioni, l'aggiornamento potrebbe degenerare e la matrice approssimata potrebbe diventare mal condizionata.

Nell'articolo di Shi et al. [1], si osserva che in presenza di errori (rumore) nella valutazione della funzione e del gradiente, il classico schema BFGS può fallire: differenze di gradiente  $y_k$  calcolate su passi troppo piccoli possono essere dominate dal rumore e corrompere l'aggiornamento. Per mitigare questo problema, gli autori propongono modifiche pratiche all'algoritmo BFGS. Gli elementi chiave dell'algoritmo BFGS tollerante al rumore sono:

- **Condizione di controllo del rumore e lengthening:** oltre ad usare le condizioni di Wolfe forti (2, 3) per determinare il passo  $\alpha_k$ , viene introdotto il parametro di lengthening (allungamento)  $\beta_k \geq \alpha_k$  e richiedono la noise control condition:

$$(\nabla f(x_k + \beta_k d_k) - \nabla f(x_k))^T d_k \geq 2(1 + c_3)\epsilon_g \|d_k\|, \quad (5)$$

dove  $c_3 > 0$  è una costante predefinita e  $\epsilon_g$  è una stima superiore del rumore sul gradiente. Questa condizione garantisce che la differenza osservata nei gradienti sia sufficientemente grande rispetto al rumore e quindi che  $y_k$  rifletta la curvatura vera e non solo il rumore. Se  $\alpha_k$  non produce un  $y_k$  affidabile, l'algoritmo allunga il passo (aumenta  $\beta_k$ ) fino a raccogliere una differenza di gradienti significativa.

- **Line search a due fasi tollerante al rumore:** la procedura proposta è una line search a due fasi, nella prima fase si cerca un  $\alpha_k$  che soddisfi contemporaneamente le condizioni di Wolfe forti (2, 3) e la noise control condition (5). In questo caso si pone  $\beta_k = \alpha_k$ . Se non si riesce a trovare tale  $\alpha_k$ , si entra nella seconda fase, in cui si cerca un  $\alpha_k$  che soddisfi la condizione di Armijo e un  $\beta_k \geq \alpha_k$  che soddisfi solo la noise control condition. In questo modo, l'algoritmo garantisce che l'aggiornamento della matrice Hessiana sia basato su informazioni affidabili, anche in presenza di rumore.
- **Aggiornamento della matrice  $\beta_k$ :** una volta determinati  $\alpha_k$  e  $\beta_k$ , l'aggiornamento della matrice Hessiana approssimata  $B_k$  viene eseguito con la stessa formula dell'algoritmo BFGS (4), ma utilizzando

$$s_k = \beta_k d_k, \quad y_k = \nabla f(x_k + \beta_k d_k) - \nabla f(x_k).$$

Questo assicura che l'aggiornamento sia basato su una differenza di gradienti che supera il rumore, migliorando la stabilità e l'affidabilità dell'algoritmo in contesti rumorosi.

Il contributo principale del paper non è cambiare la struttura fondamentale dell'algoritmo BFGS, ma rendere robusto l'algoritmo esistente in presenza di rumore, introducendo meccanismi per garantire che le informazioni utilizzate per aggiornare la matrice Hessiana siano affidabili. Queste modifiche permettono a BFGS tollerante al rumore di conservare i vantaggi dei metodi Quasi-Newton, anche quando le valutazioni sono affette da errori.

### 3 Esperimenti

In questa sezione vengono presentati gli esperimenti volti a valutare le prestazioni dell'algoritmo BFGS tollerante al rumore, confrontandolo con BFGS classico (sia nella versione implementata personalmente sia nella versione fornita dalla libreria SciPy) e con il metodo di discesa del gradiente implementato personalmente. Gli esperimenti sono stati eseguiti su un benchmark di sei problemi presi dalla libreria PyCUTEst. Per ciascun metodo è stato registrato: il numero di iterazioni necessarie per la convergenza (definita come norma del gradiente inferiore a un valore tolleranza), l'andamento dei valori della funzione obiettivo e del gradiente ad ogni iterazione, e il tempo di esecuzione totale. Per gli algoritmi progettati per problemi non rumorosi abbiamo valutato le prestazioni sia su funzioni prive di rumore sia su funzioni rumorose, in modo da evidenziare l'effetto del rumore sugli algoritmi classici e successivamente verificare l'efficacia della versione BFGS tollerante al rumore. Nei paragrafi successivi sono riportati i dettagli i risultati ottenuti, presentati sia in forma tabellare che grafica.

Prima di passare ai risultati, viene fornita una breve panoramica dei dettagli tecnici degli esperimenti:

- **Ricerca di linea:** per il metodo di discesa del gradiente (GD) è stata implementata una ricerca di linea che utilizza la sola *condizione di Armijo*. Per BFGS classico è stata implementata una ricerca di linea che soddisfa le *condizioni di Wolfe forti*. Per BFGS tollerante al rumore la ricerca di linea è basata sulle stesse Condizioni di Wolfe forti, con l'aggiunta della *condizione di controllo del rumore* descritta nella Sezione 2.
- **Aggiunta di rumore:** ad ogni valutazione della funzione obiettivo e del gradiente è stato aggiunto rumore gaussiano  $\mathcal{N}(0, \sigma^2)$ , dove  $\sigma$  è il livello di rumore scelto per l'esperimento. Alla valutazione scalare della funzione obiettivo si somma un termine scalare di rumore; al gradiente si aggiunge invece un vettore di rumore, perturbando ciascuna componente.
- **Tolleranza:** una volta fissati i livelli di rumore (in particolare quello sul gradiente), la tolleranza per la condizione di arresto è stata determinata in modo dinamico affinché risultasse sempre maggiore del rumore sul gradiente. In questo modo si evita che il rumore impedisca la terminazione degli algoritmi. Inoltre, per garantire la coerenza nel confronto tra metodi, la stessa regola è stata applicata anche alle istanze prive di rumore: la tolleranza è stata calcolata come se fosse presente rumore, mantenendo omogenee le condizioni di arresto tra gli esperimenti.

#### 3.1 Numero di Iterazioni

#### 3.2 Andamento Funzione Obiettivo e Norme del Gradiente

#### 3.3 Tempi di Esecuzione

**Tabella 1:** Numero di iterazioni per la convergenza dei diversi metodi sui problemi di benchmark, sia in assenza che in presenza di rumore.

| Problema | GD<br>(no rumore) | GD<br>(con rumore) | BFGS<br>(no rumore) | BFGS<br>(con rumore) | SciPy<br>(no rumore) | BFGS<br>(con rumore) | SciPy<br>(con rumore) | BFGS<br>(con rumore) | Tollerante<br>(con rumore) |
|----------|-------------------|--------------------|---------------------|----------------------|----------------------|----------------------|-----------------------|----------------------|----------------------------|
| ALLINITU | X                 | Y                  | Z                   | A                    | B                    | C                    | C                     | D                    | D                          |
| ARWHEAD  | X                 | Y                  | Z                   | A                    | B                    | C                    | C                     | D                    | D                          |
| BOX      | X                 | Y                  | Z                   | A                    | B                    | C                    | C                     | D                    | D                          |
| BOXPOWER | X                 | Y                  | Z                   | A                    | B                    | C                    | C                     | D                    | D                          |
| BRKMCC   | X                 | Y                  | Z                   | A                    | B                    | C                    | C                     | D                    | D                          |
| BROYDN7D | X                 | Y                  | Z                   | A                    | B                    | C                    | C                     | D                    | D                          |

## Riferimenti bibliografici

- [1] Hao-Jun M Shi, Yuchen Xie, Richard Byrd, and Jorge Nocedal. A noise-tolerant quasi-newton algorithm for unconstrained optimization. *SIAM Journal on Optimization*, 32(1):29–55, 2022.